Obfuscating Game Hacking Through Hardware

Yannis Lagrosa, Bryan Deason, Braell Dotson, Lamar Dave

Department of Computer Science, Sam Houston State University, Huntsville, Texas
Professor, Dr. Cihan Varol

Abstract

Video-game hacking is a field that is constantly evolving. There exists a perpetual war between defenders and threat actors and as new technologies emerge, this battle is fought on more fronts than ever. Through a case study, this research document will analyze and apply developmental technology in the game hacking field to the game Deadlock. This will serve as a penetration test for the game, dissecting the game's internal structures and using various memory manipulation tools to develop a software that grants users a competitive edge over the regular player. This document aims to demonstrate the capability of these technologies as well as discuss potential solutions that Valve, the game developers, could consider when developing their anti-cheat program. Ultimately, this document aims to foster a fair online gaming environment.

Keywords: videogame hacking, reverse engineering, DMA attack, IDA, windows internals

---

1 Introduction

1.1 Overview

1.1.1 Definition

Game Hacking can be defined as abusing the game's logic to exploit the system and acquire a competitive edge over others.

1.2 Why is it important?

Companies spend millions of dollars developing videogames and anticheats [1] to secure servers and ensure that every player of their game abides by the rules determined by the creators.

Cheats which are developed typically by malicious third parties potentially cost companies millions of dollars in revenue. Depending on the scale of the cheat, it may even be tortious, and thus companies may even sue cheat developers. [2]

Although videogames generally aren't thought of as needing to be secured, negligence to this will undoubtedly allow cheaters in the game to run rampant, potentially even reducing the game's lifespan as the common player acknowledges the unfair advantage that others seem to have.

1.3 Key Terms

Game hacking, which holds heritage in malware development, is essentially its own sphere in the Computer Science domain. Because of this, there are many terms common in game hacking which are rarely found in the rest of computer science. We will specifically use three techniques that all computer scientists may not be aware of.

Term	Definition
Offset	An offset is a distance from the start of a segment of memory to a certain variable.
Array of Bytes	Sequential hexadecimal values denoting the Assembly Operation Codes found in the binary of the process.

1.4 Literature Review

Looking at other works in the field may give us more insight on how to hack Deadlock, and the tools we may need to use.

Table 1: Related Works.

Authors(Year)	Focus of Study	Key Findings	Limitations
Quang Hoang William & Mary (2025)	Game Hacking	Case study on how to hack Assault Cube	Only speculated on an anti-cheat since Assault Cube did not have one.
Iantorno, Michael (2019)	Videogame Hacking	Identified videogame hacking practices and methodologies.	Did not discuss the utilization of hardware.
Caroline Andrea Rendenbach (2022)	Anti-Cheat Software	Dissected various anti-cheat technologies and their implementation in games.	Did not tackle usage of machine-learning in anti-cheats.
Patrick Stewin, Iurii Bystrov (2012)	DMA Attack	Explained and tested the feasibility of DMA-based Malware attacks.	Did not mention other cybercrimes possible through this technology. How could it affect Game Hacking or similar fields?

1.5 Motivation

With the ever-growing popularity of e-sports, cheating has been difficult to monitor and avoid. This document hopes to cultivate a more positive environment for the currently invite-only game, Deadlock, by hacking and writing a detailed report on the vulnerabilities used. Ideally, this would allow the game to have a smooth transition into Open Beta and then Public Release.

This document will use industry grade tools and methodologies to give a realistic scenario on how hackers might attempt to develop cheats for the game, bypassing forms of anti-cheat [3]. By implementing this into a real game, averaging a concurrent 100,000 players per day, we can demonstrate architectural errors in even multi-billion dollar companies such as Valve.

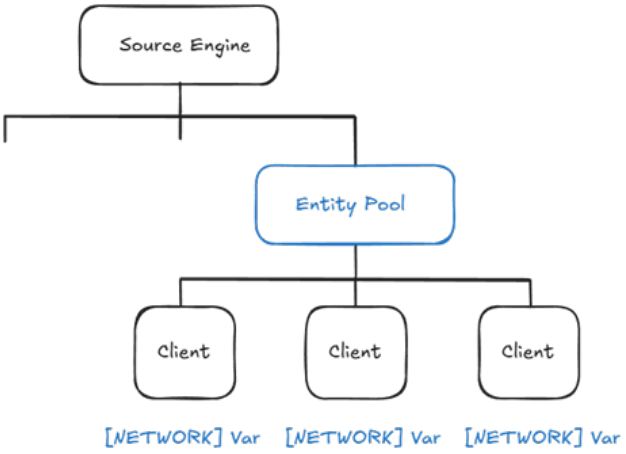


Figure 1: Source Engine Networked Variables Diagram.

2 Methodology

In order to begin hacking the game, we need to understand the architecture of the engine the game is built on. Deadlock is built on the Source 2 engine. But before looking at Source 2, we should look at how its predecessor, the Source Engine, handles Networked Variables. The server propagates through all valid clients and updates "Network variables" when the variable is changed on the server so that all clients remain homogenous. What can be noted about this, is that in order for Networked variables to work properly, the client must know where the variables are, shown in *Figure 1.* By the philosophy of this design, the offsets to these variables are revealed to the client, so that the client can determine which variable to update. [4] In the Source 2 Engine, the network variable procedure is modified by the means of a new feature called the Schema System. This Schema System will bind classes, enumerators, and types to a schema listing their variables, and most importantly offsets. Valve implements this on the client-side through a DLL that is required by the game, schemasystem.dll, which provides functions for the client to generate these schema.

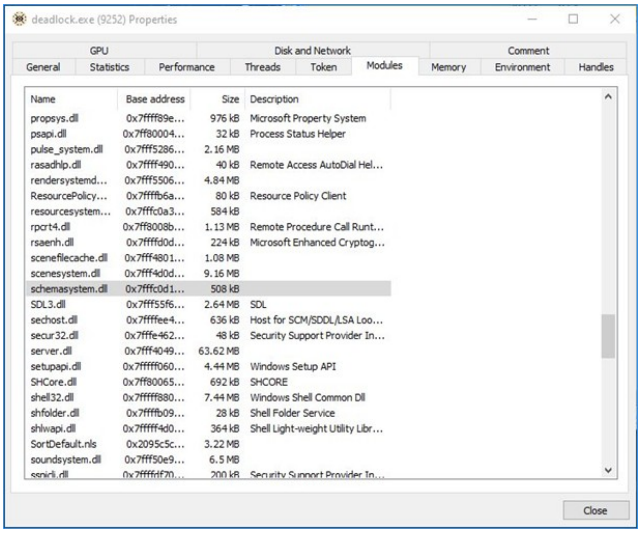


Figure 2: schemasystem.dll at runtime, displayed using Process Hacker.

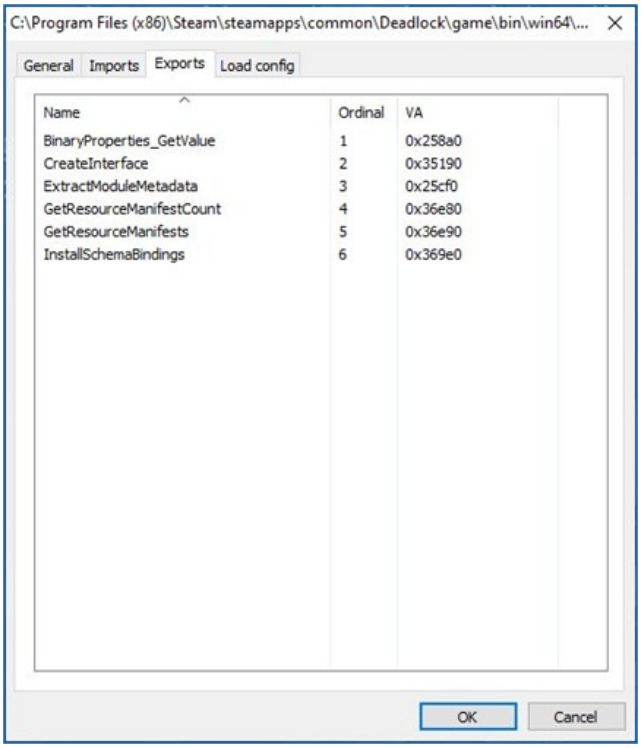


Figure 3: schemasystem.dll exports CreateInterface function

2.1 Creating an SDK for the game

We can use this knowledge of the schemasystem.dll to create an SDK of the game, essentially calling the schema to generate offsets and patterns for various classes in the game.

Using a debugger, x64Dbg in our case, we set a breakpoint on the CreateInterface function before the process launches to see what parameters might be required for it.



Figure 4: schemasystem.dll exports CreateInterface function

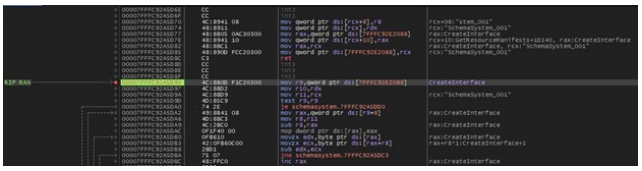


Figure 5: schemasystem.dll exports CreateInterface function

Analyzing the stack, we see that CreateInterface could be constructed using the following line of code:

```
SchemaSystem* schemaSystem = CreateInterface("
    SchemaSystem_001", nullptr);
```

Now that we understand the required syntax for this function, we can create a DLL to inject into the game, which will call this function from the deadlock.exe process.

Basically, how this works, is we create a .dll file that calls the CreateInterface function, using the necessary parameters. In order to execute this function in the game's process, we first allocate memory to deadlock.exe using the WinAPI function VirtualAllocEx Now that we understand the required syntax for this function, we can create a DLL to inject into the game, which will call this function from the deadlock.exe process.

Basically, how this works, is we create a .dll file that calls the CreateInterface function, using the necessary parameters. In order to execute this function in the game's process, we first allocate memory to deadlock.exe using the WinAPI function VirtualAllocEx

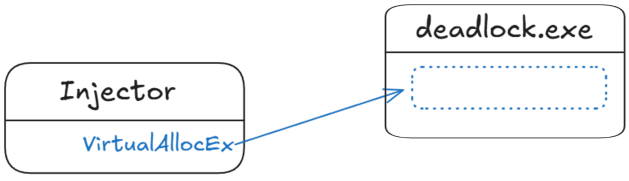


Figure 6: Using VirtualAllocEx to allocate space to Deadlock.exe

Then, we use WriteProcessMemory to write the path of our file into that space.

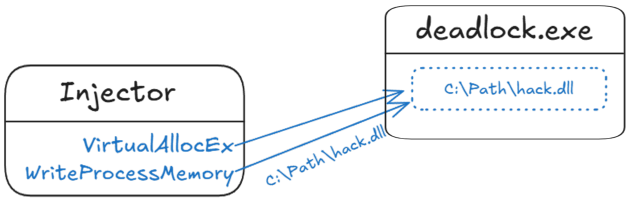


Figure 7: Using WriteProcessMemory to write the .dll name to deadlock.exe

Finally, we can create a remote thread that calls LoadLibraryA, loading our DLL into the game.

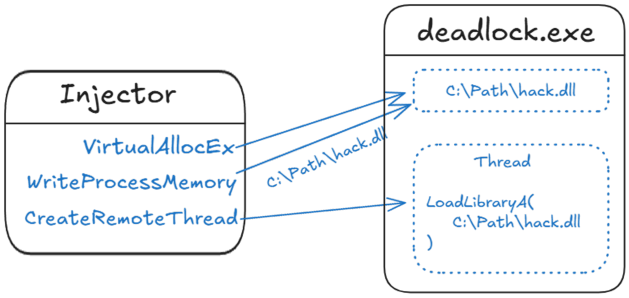


Figure 8: Using CreateRemoteThread and LoadLibraryA to execute the .dll file from inside the deadlock.exe process

This yields a collection of both header and body C++ files. These files are detailed with the offsets to local variables, which we will be able to access and utilize in the construction of our cheat. This is colloquially known as a dump of the game or by software engineers- an SDK.

2.2 Finding the Entity List

The next task required is to find the EntityList, which is not generated in the SDK. We can achieve this by scanning the client.dll in IDA for an Array of Bytes. By scanning through the memory of the game and identifying various health values for our local player during an instance of the binary, we can analyze the process to identify assembly calls that access our health value, *Figure 9.*

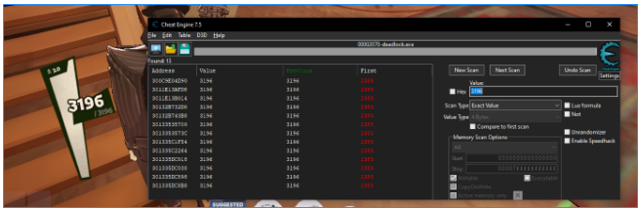


Figure 9: Scanning for player health.

We then remove the offset to find our base pointer to the player, *Figure 10.*

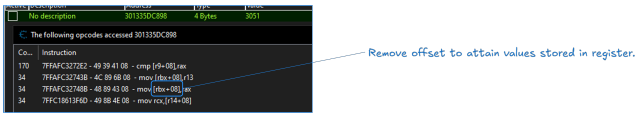


Figure 10: Removing offset from register to locate base entity address.

After scanning for our Entity List, we can dereference the values and build an algorithm to iterate through each of the entities.

In the case of Deadlock, there is a padding of 0x70 between each CCitadelPlayerController, which is a class that holds the player objects and information such as their position, *Figure 11.*

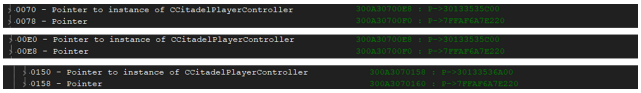


Figure 11: CCitadelPlayerController objects located in memory using Cheat Engine.

2.3 Hardware

The first piece of equipment we will be using is a Screamer PCIe Squirrel by Lambda Concept.

A DMA or Direct Memory Access Device is a piece of hardware that bypasses the kernel and directly interfaces with DRAM of the system. [5] What this allows us to do is read memory without the usage of any WinAPI calls, which can be monitored by anti-cheats for suspicious activity [6], *Figure 12.*

Now, for implementing the board, we can simply flash it with the necessary gateware and drivers and then use an SDK such as PCILeech to read pages of memory from our target.

Now that the DMA is properly configured, we can read information from the game using a simple C++

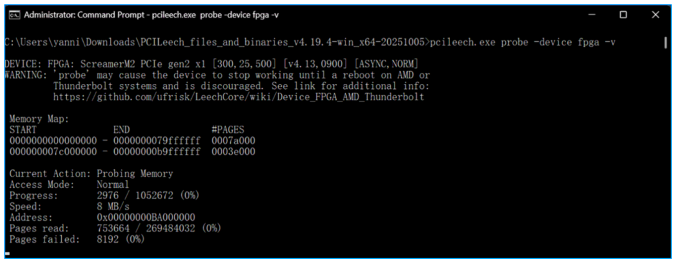


Figure 12: Probing device using PCILeech.

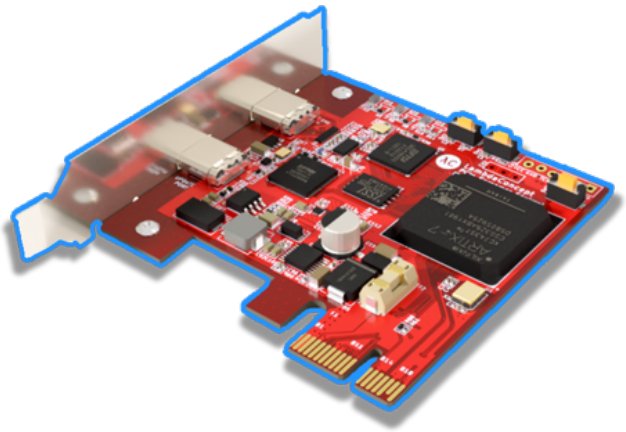


Figure 13: PCIe Squirrel.

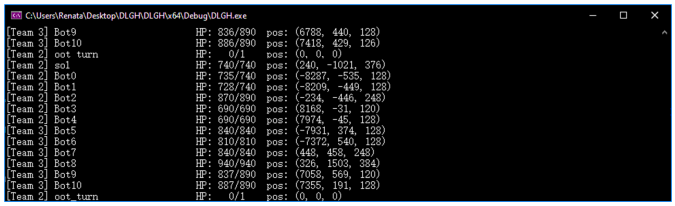


Figure 14: Entity List in console.

program, and format it, printing out our entity list with each of the player's positions, *Figure 14*.

2.4 Minimap Program

The next step is to build a program to plot all of the players on our version of the game map. To develop the external map, we used a Java program along with OpenGL to produce a dynamic output capable of taking live information and using it to draw shapes in real time.

The program uses a loop that plots the players on our map, using information taken from the scanned memory, and sent via a webserver. Where it is then drawn by the Java program and OpenGL.

To set up OpenGL, we used IntelliJ IDEA with

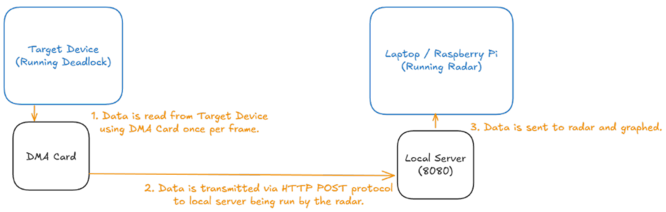


Figure 15: Diagram of external map.

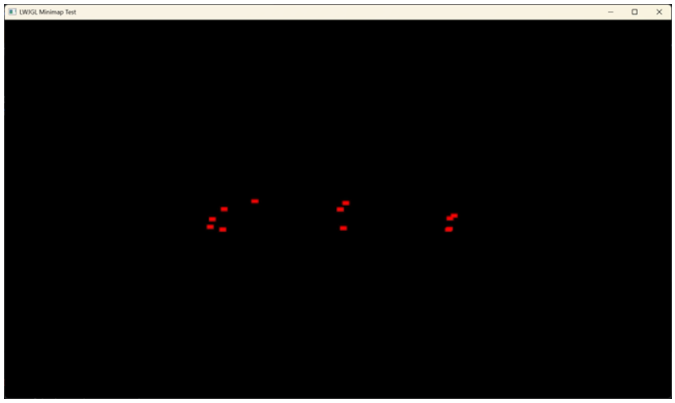


Figure 16: Radar running on PC 2.

the LWJGL framework. After importing the required libraries and configuring the native library path, we were able to create a basic rendering setup capable of opening a window and drawing graphics in real time.

After information is gathered from the game via the DMA card, it is sent via the webserver, and parsed through using RegEx to rebuild the player state for each player. The player positions are then normalized by the game map size, *Figure 16*.

3 Results and Discussion

We were able to reverse engineer the game, identify offsets for players, and externalize our cheat using a DMA card and a second PC. [7] Each of these three tasks demonstrate flaws in the security of the videogame, which should ideally be patched or at least made more difficult before the game's release.

Our analysis of the program helped our team identify several key vulnerabilities that the game currently has, *Table 2*.

4 Conclusion

Even though the goal of our project was to help improve the game by exposing these vulnerabilities, ultimately cheat developers and game developers will always be at war with one-another. It will be impossible to guarantee a completely neutral game environment. It is still within the best interest of Valve to improve upon their anti-cheat though. If cheating requires set-up such as hardware, it intrinsically becomes more exclusive, meaning there will be less cheaters on the servers.

Table 2: Key findings.

Vulnerability	How we manipulated it	Suggestions to fix it)
SchemaSystem reveals all offsets to client.	This vulnerability allowed our team to analyze the memory of the game and reveal information stored only in the code.	Improve injected DLL detection via an anti-cheat software or module. Monitor process memory.
Lack of Reverse Engineering counter-measures	Our team was able to identify critical memory addresses used to develop the cheat.	Monitor WinAPI function calls. Terminate the process if Windows attempts to debug it.
Lack of Peripheral Monitoring	Our team was able to use a DMA card to read memory from the game.	Monitor peripherals, especially the PCIe port. Scan for known DMA firmware patterns/strings.

We hope to share our findings with the creators of this game to improve its development cycle.

5 Future Work

In the future, we hope to continue this document, adding various features such as a height indicator for the external map, having the external map display player icons, and overlaying the external map onto the minimap in-game using a fuser.

References

[1] e. a. Sam Collins, "Anti-cheat: Attacks and the effectiveness of client-side defences," *https://tomchothia.gitlab.io/Papers/AntiCheat2024.pdf*, 2024.

[2] R. Shikowitz, "License to kill: Mdy v. blizzard and the battle over copyright in world of warcraft," *https://brooklynworks.brooklaw.edu/cgi/viewcontent.cgi?ar* 2010.

[3] Ubisoft, "Fairfight anti-cheat software," *https://www.i3d.net/anti-cheat-software/*, 2021.

[4] Praydog, "An analysis of the source 2 engine: The schema system," *https://praydog.com/reverse-engineering/2015/06/24/source2.html*, 2015.

[5] D. Aumaitre, D., "Subverting windows 7 x64 kernel with dma attac ks," *http://esec-lab.sogeti.com/dotclear/public/publications/10-hitbamsterdam-dmaattacks.pdf*, 2010.

[6] Steampowered, "Vac-enabled steam games," *https://store.steampowered.com/search/?category1=998ca*

[7] Steamworks, "Vac integration (steamworks documentation)," *https://partner.steamgames.com/doc/features/anticheat/v ¡ntegration.*